

SuperBizAgent Python 项目

整体架构、信息流与模块职责说明

单一 Durable Incident Runtime 完成态架构

文档目的	帮助新成员由浅入深理解系统边界、主流程、状态和依赖
分析范围	应用入口、前后端、工作流、数据层、外部系统、测试与运行方式
生成日期	2026-08-18
项目目录	C:\zyh\OnCallAgent\Python-super_biz_agent_py-release-2026-05-17\SpatialS RE-Agent

一、项目定位

这是一个面向值班运维人员的 AI 运维助手，主要解决告警出现以后，如何查资料、收集证据、分析根因并形成处置建议的问题。

系统提供两类入口：普通 AI 对话和知识问答保持独立；故障诊断统一进入 Durable Incident Runtime，由 Incident Router 在 Simple 与 Enterprise 两种策略之间选择。

后端主要使用 FastAPI、LangGraph、LangChain、Qwen/DashScope、PostgreSQL 和 Milvus；前端是原生 HTML、CSS、JavaScript，没有 Vue、React 等前端框架。

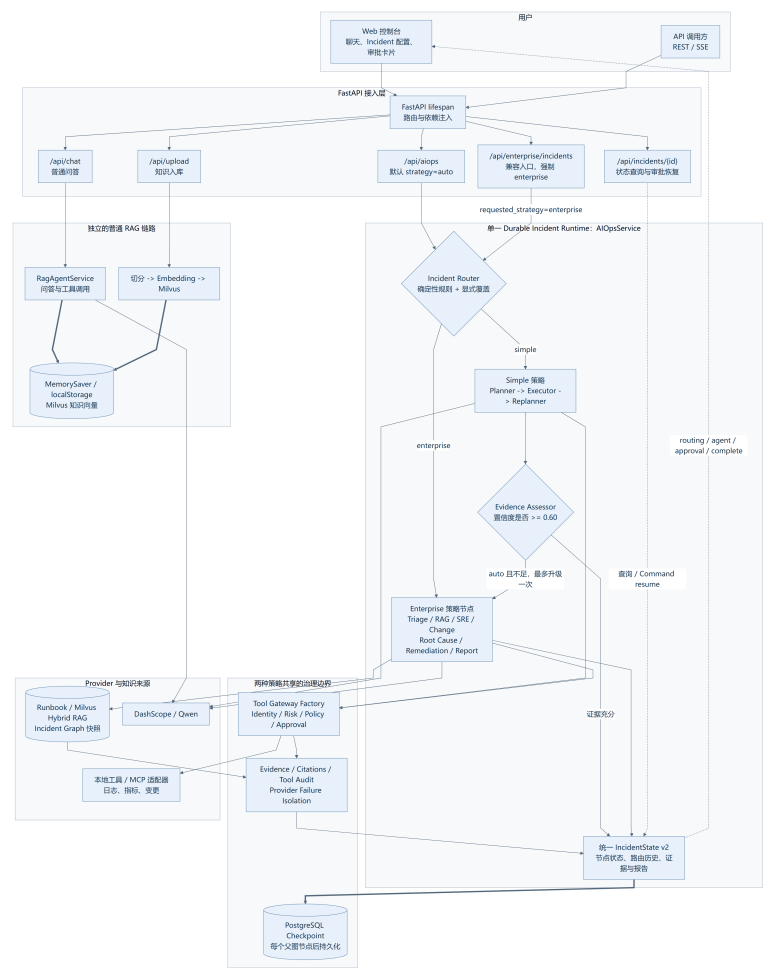
应用由 Uvicorn 启动。FastAPI 在启动阶段连接 Milvus 和 PostgreSQL，并通过 REST API 或 SSE 流向浏览器返回结果。

阅读顺序

- 先看系统总览，记住普通 RAG 独立，两个事故接口汇入同一个 AIOpsService。
- 再看聊天与知识入库，理解普通问答和 Milvus 知识库的关系。
- 然后看统一 Runtime，理解确定性路由、动态升级、审批与 PostgreSQL 恢复。
- 接着看 Enterprise 策略，理解多个角色如何并行调查并共享工具治理。
- 最后看启动与存储边界，确认哪些数据会保留，哪些只存在内存。

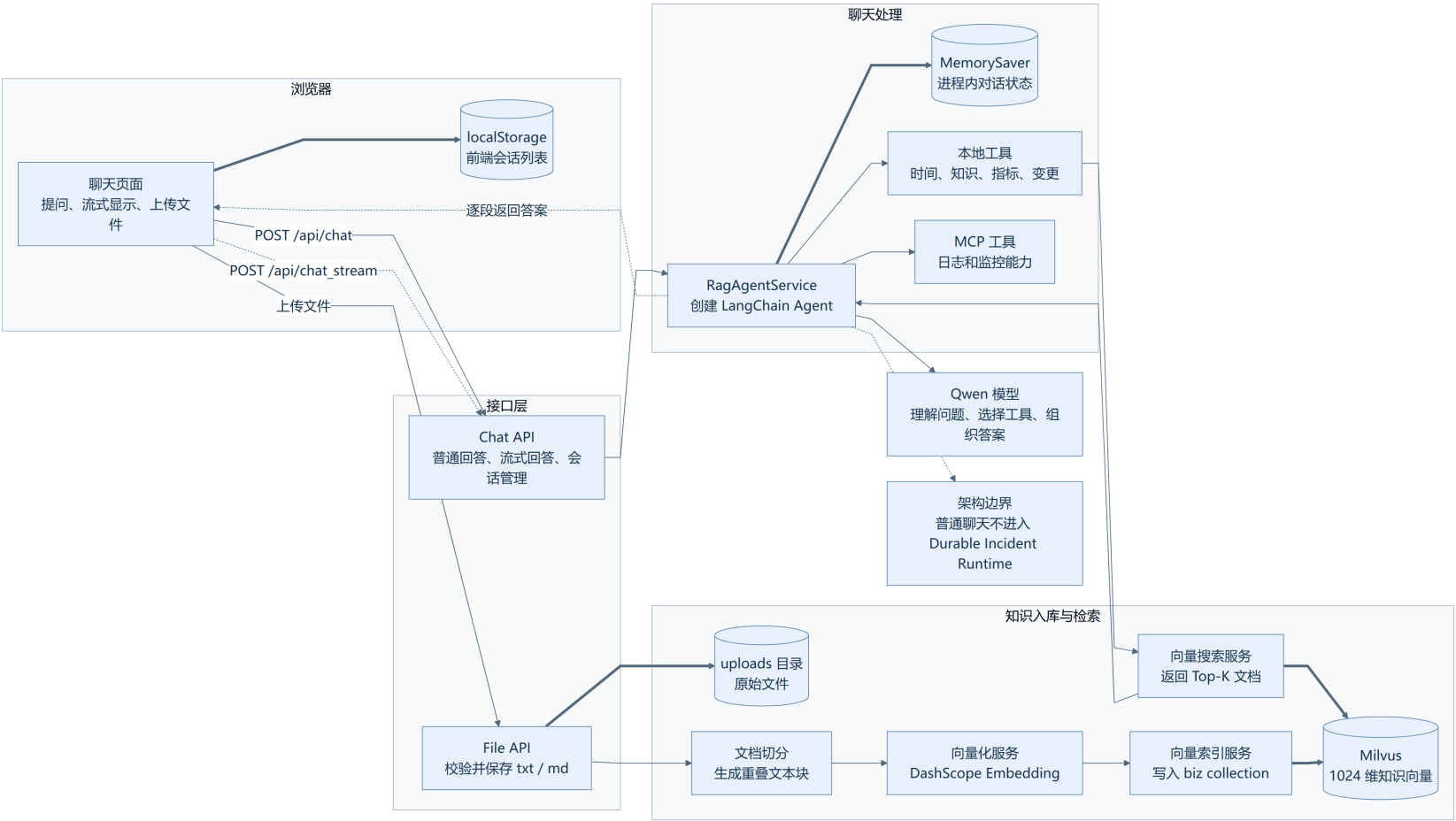
图例：实线箭头表示同步 HTTP 或程序内部调用；虚线箭头表示 SSE、并行流程或人工中断与恢复；粗线箭头表示数据库、文件或状态读写。

二、整体结构图 - 系统总览



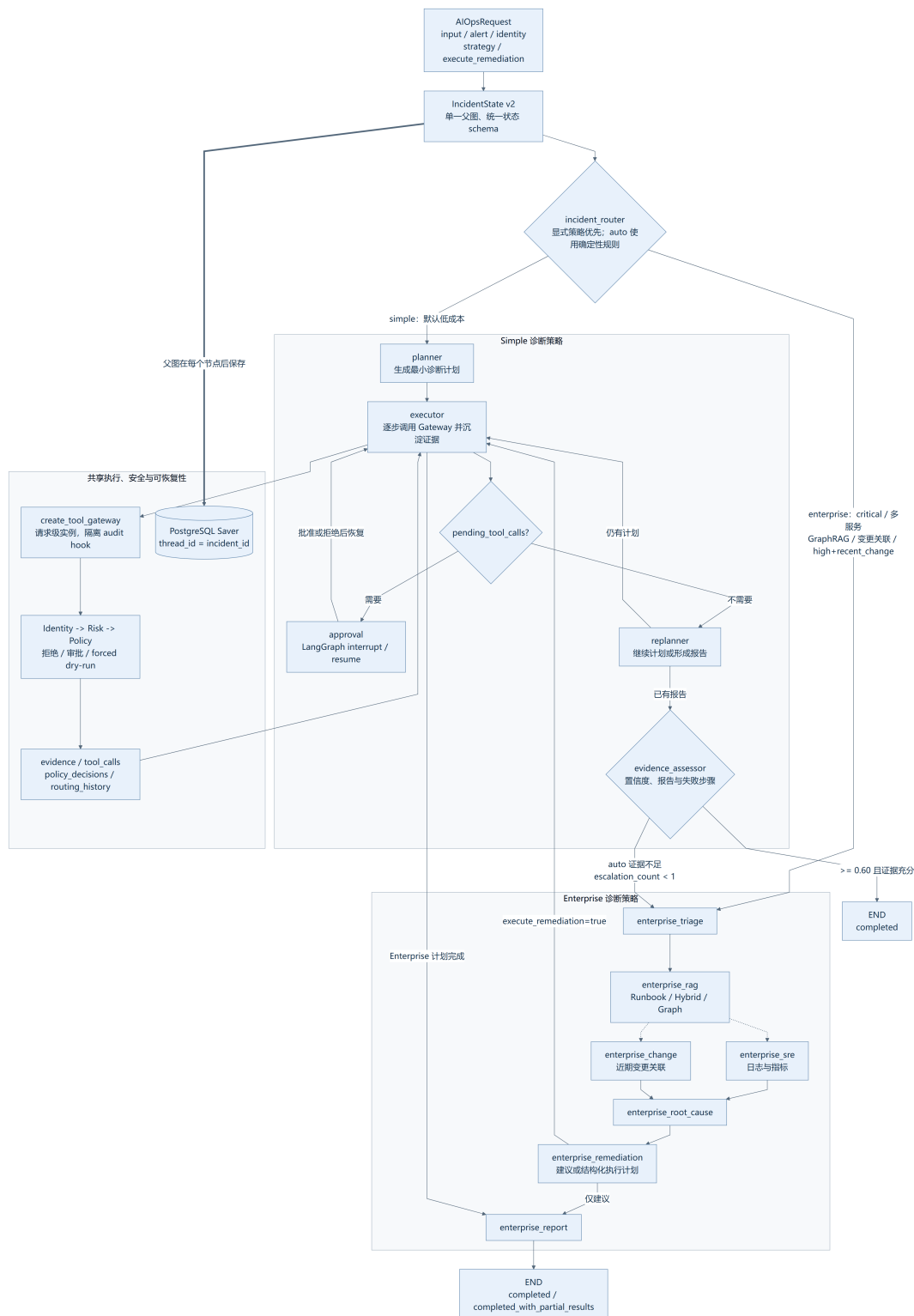
大白话理解：普通聊天是独立问答链路；所有故障请求进入同一个 AIOpsService。Router 选择 Simple 或 Enterprise，两种策略共享 IncidentState、Checkpoint、Gateway、审批和审计。

子图一：普通聊天与知识入库



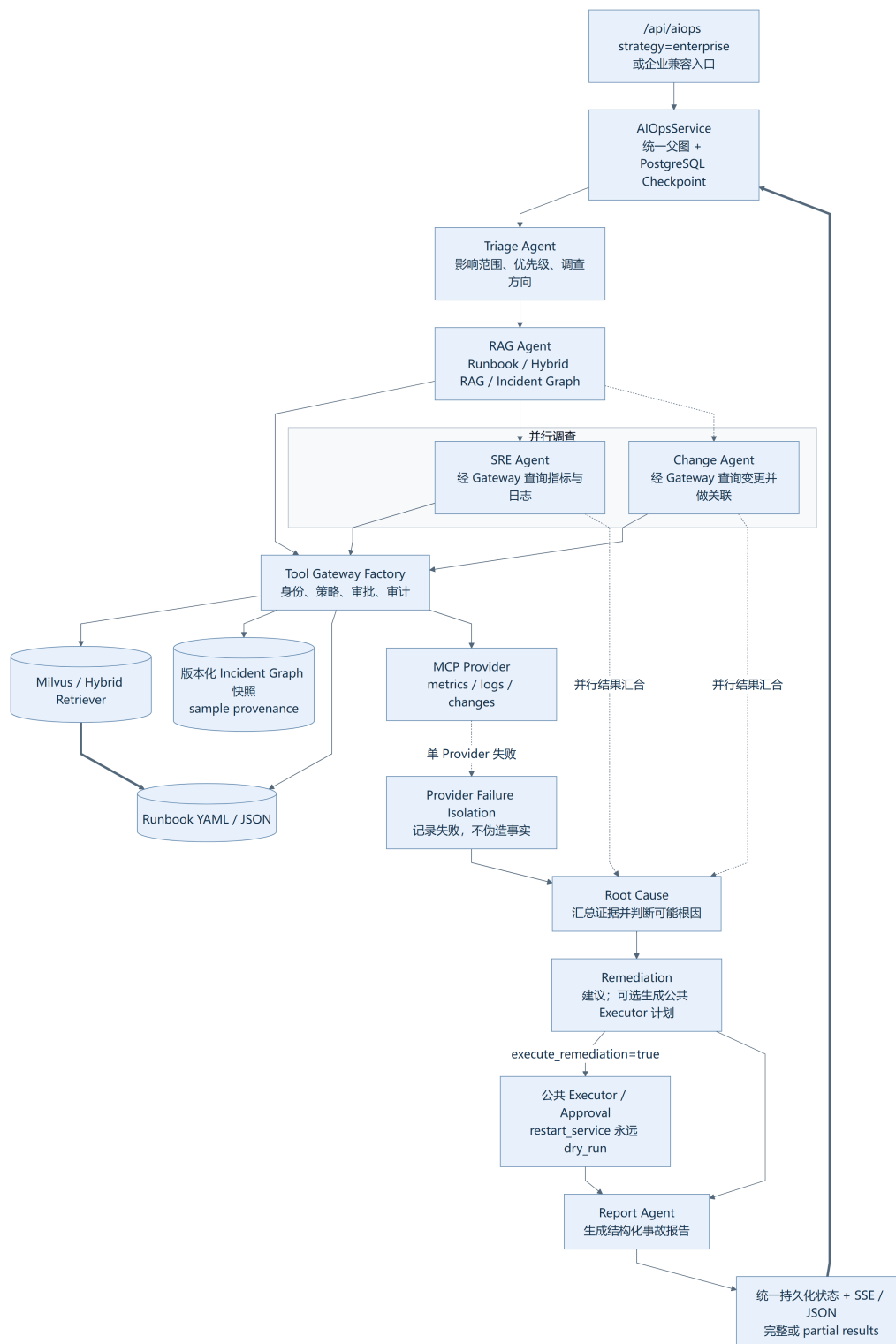
普通聊天使用进程内 MemorySaver；浏览器另存一份 localStorage 历史。上传文档经过切分与向量化后进入 Milvus。Chat Agent 直接绑定工具，不经过 Tool Gateway。

子图二：持久化 AIOps 诊断



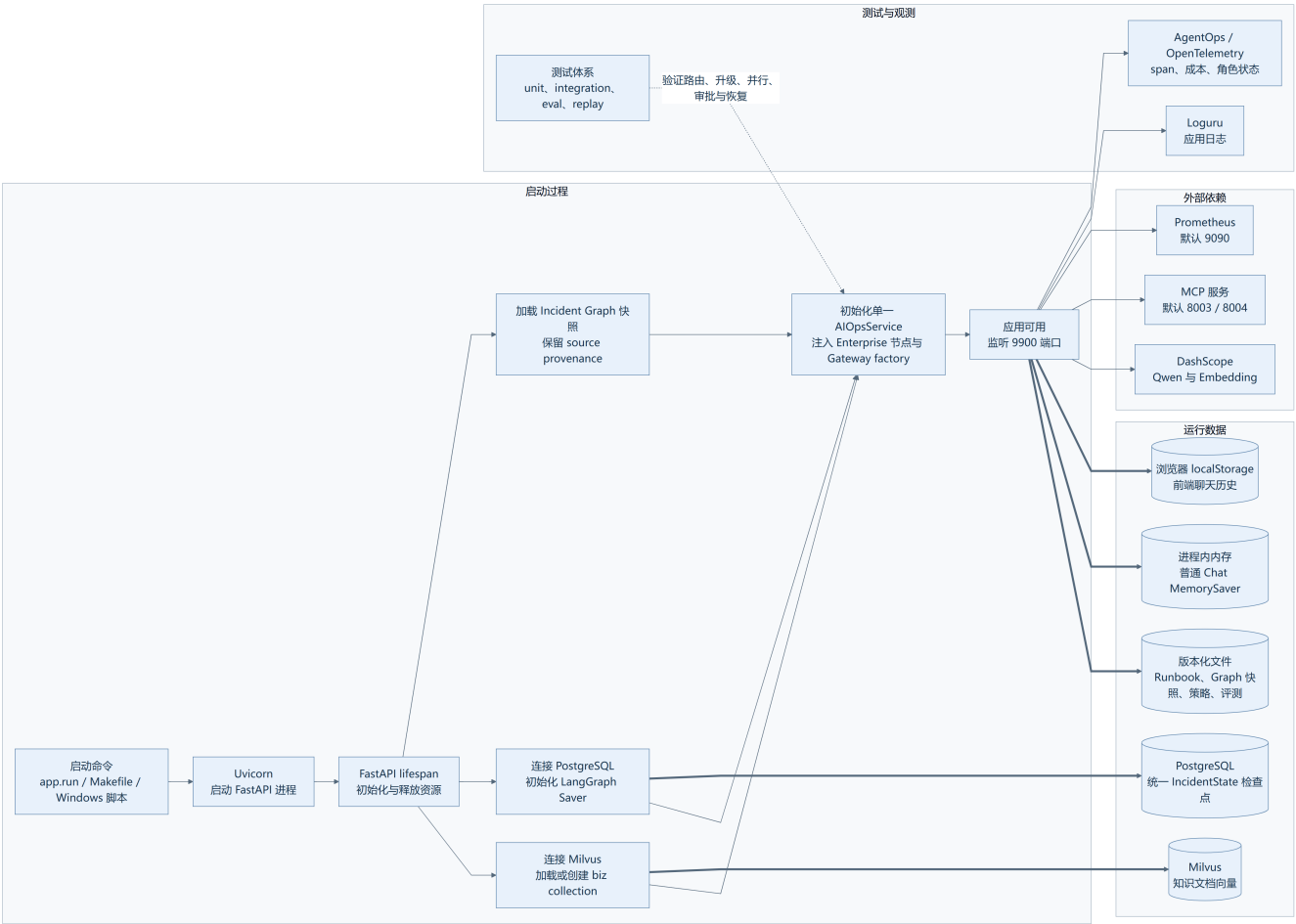
Router 先依据显式策略与故障复杂度选路。Simple 证据不足时最多升级一次；Enterprise 复用公共 Executor 和 Approval。父图在每个节点后写入 PostgreSQL，重启后按 incident_id 恢复。

子图三：企业多智能体事故分析



Enterprise 是父图中的专业诊断策略：Triage -> RAG -> SRE/Change 并行 -> Root Cause -> Remediation -> Report。外部查询统一经过 Gateway；Provider 失败被隔离并形成 partial results，不生成替代事实。

子图四：启动、存储与运行边界



FastAPI lifespan 初始化 Milvus、PostgreSQL Saver、Incident Graph 快照与单一 AIOpsService。PostgreSQL 保存两种策略的统一状态；普通聊天的 MemorySaver 与浏览器历史保持独立。

三、核心调用链

1. 普通聊天

浏览器 -> Chat API -> RagAgentService -> Qwen -> 本地或 MCP 工具 -> 返回答案

主要模块：static/app.js、app/api/chat.py、app/services/rag_agent_service.py、app/tools/、app/agent/mcp_client.py。

2. 文档进入知识库

上传文件 -> File API -> uploads 目录 -> 文档切分 -> DashScope Embedding -> Milvus

主要模块：app/api/file.py、document_splitter_service.py、vector_embedding_service.py、vector_index_service.py、vector_store_manager.py。

3. 统一 Durable Incident Runtime

AI Ops API -> Incident Router -> Simple 或 Enterprise -> 统一 IncidentState -> PostgreSQL -> SSE

显式策略覆盖自动路由；auto 对 critical、多服务、GraphRAG/变更关联和 high+recent_change 直接选择 Enterprise，其余先走 Simple。incident_id 同时作为 LangGraph thread_id。

4. Enterprise 诊断策略

Router -> Triage -> RAG -> SRE 与 Change 并行 -> Root Cause -> Remediation -> 公共 Executor/Approval（可选） -> Report

企业兼容入口强制 enterprise，但仍调用同一个 AIOpsService。Runbook、Hybrid RAG、Incident Graph、MCP Provider 和处置工具都受统一审计边界约束。

5. 应用启动

app.run -> Uvicorn -> FastAPI lifespan -> Milvus / PostgreSQL -> Incident Graph -> 单一 AIOpsService -> 开放接口

生命周期集中创建与释放连接池；EnterpriseIncidentWorkflow 只向父图注册节点，不持有独立生产运行链。

四、入口与诊断策略的关键差异

路径	主要用途	状态保存	工具治理
普通聊天	问答、知识检索	MemorySaver + 浏览器 localStorage	工具直接绑定，不经过 Tool Gateway
Simple 策略	低成本、逐步诊断	统一 IncidentState + PostgreSQL	Gateway、身份、策略、审批、审计
Enterprise 策略	跨日志、指标、变更和图谱的多角色分析	统一 IncidentState + PostgreSQL	同一 Gateway；Provider 失败隔离

- 普通聊天用于知识问答；需要持久化、审批或审计的故障请求统一进入 AIOpsService。
- Simple 优先控制成本，证据不足时由父图动态升级，已采集证据不会丢失。
- Enterprise 负责复杂故障的专业分工，并与 Simple 共享执行和安全底座。

五、模块职责表

模块/子系统	核心职责	关键目录或文件	主要依赖
应用入口	注册路由、静态资源、生命周期	app/main.py; app/run.py	FastAPI; Uvicorn
Web 前端	聊天、上传、AIOps 触发和本地历史	static/index.html; static/app.js	Fetch; SSE; localStorage
Chat API	普通和流式对话、会话查询与清理	app/api/chat.py	RagAgentService
文件 API	上传文档、触发单文件或目录索引	app/api/file.py	VectorIndexService
AIOps API	统一诊断、状态查询、审批与企业兼容入口	app/api/aiops.py	AIOpsService
Chat Agent	调用模型和工具完成普通问答	app/services/rag_agent_service.py	Qwen; MemorySaver; MCP
Durable Runtime	编译单一父图并编排双策略、审批和恢复	app/services/aiops_service.py	LangGraph; PostgreSQL
AIOps 节点	Router、Planner、Executor、Replanner、Evidence Assessor	app/agent/aiops/	Qwen; Tool Gateway
Tool Gateway	工具注册、风控、审批、超时和审计	app/agent/tool_gateway.py	Policy; Identity; Risk
证据治理	统一证据结构、引用和输入检查	app/agent/evidence.py	IncidentState
Enterprise 节点	向统一父图注册多角色诊断节点	app/agent/enterprise_workflow.py	Gateway; Hybrid RAG; GraphRAG
知识检索	向量、关键词和融合检索	vector_search_service.py; app/retrieval/hybrid.py	Milvus; 文档集合
Incident Graph	服务、事故、变更关系存储和查询	app/incident_graph/	NetworkX
Runbook	加载和匹配标准处置流程	app/runbooks.py	YAML; JSON
变更智能	关联事故与近期发布变更	app/change_intelligence.py	JSONL
PostgreSQL 检查点	保存可暂停、可恢复的事故状态	app/core/checkpoint.py	psycopg; LangGraph Saver
Milvus 管理	管理连接、Collection 和向量索引	app/core/milvus_client.py	pymilvus
评测与回放	验证检索质量和历史故障行为	app/evals/; app/replay/; tests/	pytest; JSON 数据

六、架构说明

分层说明：app/api 负责协议适配；AIOpsService 持有唯一事故父图；app/agent 提供策略节点；Tool Gateway 形成外部副作用边界；checkpoint.py 与向量服务承担持久化和数据访问。依赖始终由接入层指向运行时、策略和基础设施。

模块划分原则

普通聊天按问答场景独立；故障处理按单一 Runtime 划分。Simple 与 Enterprise 是同一父图中的可路由策略，不是两套服务。

核心依赖方向

正常方向为：浏览器或 API -> 路由 -> Service/Workflow -> 工具与检索 -> 数据库或外部系统。API 层主要负责参数接收、依赖获取、错误转换和流式返回。

关键数据流

聊天消息保存在 localStorage 与 MemorySaver；知识文档进入 Milvus；所有故障状态、路由历史、证据、审批与 Agent 输出按 incident_id 写入 PostgreSQL。

系统边界

项目内部负责 AI 编排、状态、检索和工具治理。Qwen、MCP、Prometheus、PostgreSQL、Milvus 属于外部运行依赖。系统没有 Redis、Kafka、RabbitMQ 等缓存或消息队列。

解耦边界与设计取舍

- AIOpsService 是唯一事故编排所有者；EnterpriseIncidentWorkflow 只提供节点，避免双图状态漂移。
- Gateway 通过 factory 创建请求级实例，策略注册共享、audit hook 隔离，避免并发事故串审计。
- SRE、Change 与 RAG 的外部查询统一经过 Gateway；Provider 失败写入状态而不伪造证据。
- 普通聊天保留轻量工具链，不承担生产故障执行；这一边界通过独立 API 和状态存储体现。
- restart_service 被定义为 write 风险；production 需要审批且执行层强制 dry_run。
- 旧 planner/executor/replanner/approval 节点名保留，v1 Checkpoint 可按原 next node 恢复。

七、依据与运行边界

关键代码依据

- app/main.py：应用生命周期、路由与服务初始化。
- app/api/aiops.py：AIOps、事故查询、人工审批和企业接口。
- app/services/aiops_service.py：持久化 workflow 拓扑与恢复方式。

- app/agent/tool_gateway.py: 工具治理边界。
- app/agent/enterprise_workflow.py: 企业多智能体流程。
- app/services/rag_agent_service.py: 普通聊天、MemorySaver 和工具绑定。
- app/core/checkpoint.py: PostgreSQL LangGraph 检查点。
- static/app.js: 前端请求、SSE 和 localStorage。
- docs/learning/README.md: 能力课程和验收索引。

运行边界声明

- 仓库交付本地与 Docker Compose 运行方式，不绑定 Kubernetes、虚拟机或特定云厂商。
- MCP 服务是协议适配器；日志、指标与变更的权限范围由部署环境配置。
- Incident Graph 快照带 source=sample provenance，报告能够区分样例来源与真实 Provider 证据。
- OpenTelemetry 与 AgentOps 记录 span、成本和角色状态；Exporter 由部署环境注入。
- PostgreSQL 负责 LangGraph 检查点；备份、高可用和灾备属于数据库基础设施职责。

验证记录：Python 3.13.15 下全量 pytest 132 passed，代码覆盖率 66.42%；PostgreSQL 18.6 集成测试覆盖连接池重建、事故隔离和中断后不重复节点恢复。